

**VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**



**WEB APPLICATION DEVELOPMENT
IT093IU
LAB 3**

Member List:

- Huỳnh Ngọc Bảo Long – ITITWE23023

Instructor: MSc. Nguyễn Trung Nghĩa

I. Task 2.1

- fetchWeather(city)

- Fetches current weather data using fetch() from OpenWeatherMap.
- Builds the API URL with the city and API key.
- Uses await to get the data and returns it as JSON.
- If the city doesn't exist (404), it throws an error caught in catch(), showing an error message.

```
async function fetchWeather(city) {  
  // TODO: Fetch current weather from API  
  try {  
    const response = await fetch(  
      `https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${API_KEY}&units=  
    );  
    if (!response.ok) throw new Error('City not found');  
    const data = await response.json();  
    return data;  
  } catch (error) {  
    showError(error.message);  
    return null;  
  }  
}
```

- fetchForecast(city)

- Fetches the 5-day / 3-hour forecast.
- Filters the data later to pick one result per day (at noon).
- Returns JSON data; errors are handled similarly.

```
async function fetchForecast(city) {  
  // TODO: Fetch 5-day forecast from API  
  try {  
    const response = await fetch(  
      `https://api.openweathermap.org/data/2.5/forecast?q=${city}&appid=${API_KEY}&unit  
    );  
    if (!response.ok) throw new Error('Forecast not available');  
    const data = await response.json();  
    return data;  
  } catch (error) {  
    showError(error.message);  
    return null;  
  }  
}
```

- displayWeather(data)

- Displays the current weather.
- Extracts temperature, weather condition, humidity, and wind.

- Chooses a suitable emoji icon (☀️, ☁️, 🌧️, etc.).
- Updates the HTML dynamically to show weather info.

```
function displayWeather(data) {
  // TODO: Display current weather
  // data.main.temp, data.weather[0].description
  // data.main.humidity, data.wind.speed
  const weatherDisplay = document.getElementById('weatherDisplay');
  const weather = data.weather[0].main;
  const icons = { Clear: '☀️', Clouds: '☁️', Rain: '🌧️', Snow: '❄️', Drizzle: '🌧️', Thun
  const icon = icons[weather] || '☁️';

  weatherDisplay.innerHTML = `
    <div class="weather-card">
      <div class="current-weather">
        <div>
          <h2>${data.name}, ${data.sys.country}</h2>
          <p>${data.weather[0].description} ${icon}</p>
          <p class="temp-display">${Math.round(data.main.temp)}°C</p>
        </div>
      </div>
      <p>Humidity: ${data.main.humidity}% &nbsp;&nbsp;&nbsp; Wind: ${data.wind.speed} km/h</p>
    </div>
  `;
}
```

- displayForecast(data)
 - Displays 5 days of forecast.
 - Filters forecast entries for "12:00:00" (midday).
 - Maps each to a small card showing the weekday, emoji, and temperature.
 - Creates a grid of forecast items.

```

function displayForecast(data) {
  // TODO: Display forecast (filter for one per day)
  const forecastDisplay = document.getElementById('forecastDisplay');
  const dailyData = data.list.filter(item => item.dt_txt.includes('12:00:00'));
  const icons = { Clear: '☀️', Clouds: '☁️', Rain: '🌧️', Snow: '❄️', Drizzle: '🌧️', Thun
  forecastDisplay.innerHTML = `
    <div class="weather-card">
      <h2>5-Day Forecast</h2>
      <div class="forecast-grid">
        ${dailyData.map(day => {
          const date = new Date(day.dt_txt);
          const weekday = date.toLocaleDateString('en-US', { weekday: 'short' });
          const weather = day.weather[0].main;
          const icon = icons[weather] || '☀️';
          return `
            <div class="forecast-item">
              <p>${weekday}</p>
              <p style="font-size:30px;">${icon}</p>
              <p>${Math.round(day.main.temp)}°C</p>
            </div>
          `;
        }).join('')}
      </div>
    </div>
  `;
}

```

Ln 139, Col 59 Spaces: 4 UTF-8 CRLF { } HTML Signed out Port

- searchWeather()
 - Triggered by clicking “Search” or pressing Enter.
 - Gets city name from input.
 - Displays a “Loading...” message while fetching data.
 - Calls both fetchWeather() and fetchForecast().
 - If both succeed, shows results and saves the search.

```

async function searchWeather() {
  // TODO: Get city input and fetch weather
  // TODO: Handle loading state
  // TODO: Handle errors
  // TODO: Save to recent searches
  const city = document.getElementById('cityInput').value.trim();
  if (!city) return;

  clearError();
  document.getElementById('weatherDisplay').innerHTML = '<div class="loading">Loading...</div>';
  document.getElementById('forecastDisplay').innerHTML = '';

  const weatherData = await fetchWeather(city);
  if (!weatherData) return;

  const forecastData = await fetchForecast(city);
  if (!forecastData) return;

  displayWeather(weatherData);
  displayForecast(forecastData);
  saveRecentSearch(city);
}

```

- saveRecentSearch(city)
 - Saves the searched city to localStorage.
 - Keeps a list of up to 5 most recent cities.
 - Prevents duplicates by removing and re-adding the latest city.

```

function saveRecentSearch(city) {
  // TODO: Save to localStorage (max 5 cities)
  let recent = JSON.parse(localStorage.getItem('recentCities')) || [];
  if (recent.includes(city)) {
    recent = recent.filter(c => c !== city);
  }
  recent.unshift(city);
  if (recent.length > 5) {
    recent.pop();
  }
  localStorage.setItem('recentCities', JSON.stringify(recent));
  loadRecentSearches();
}

```

- loadRecentSearches()
 - Reads cities from localStorage and shows them as clickable tags.
 - Clicking a city re-triggers a search for it.

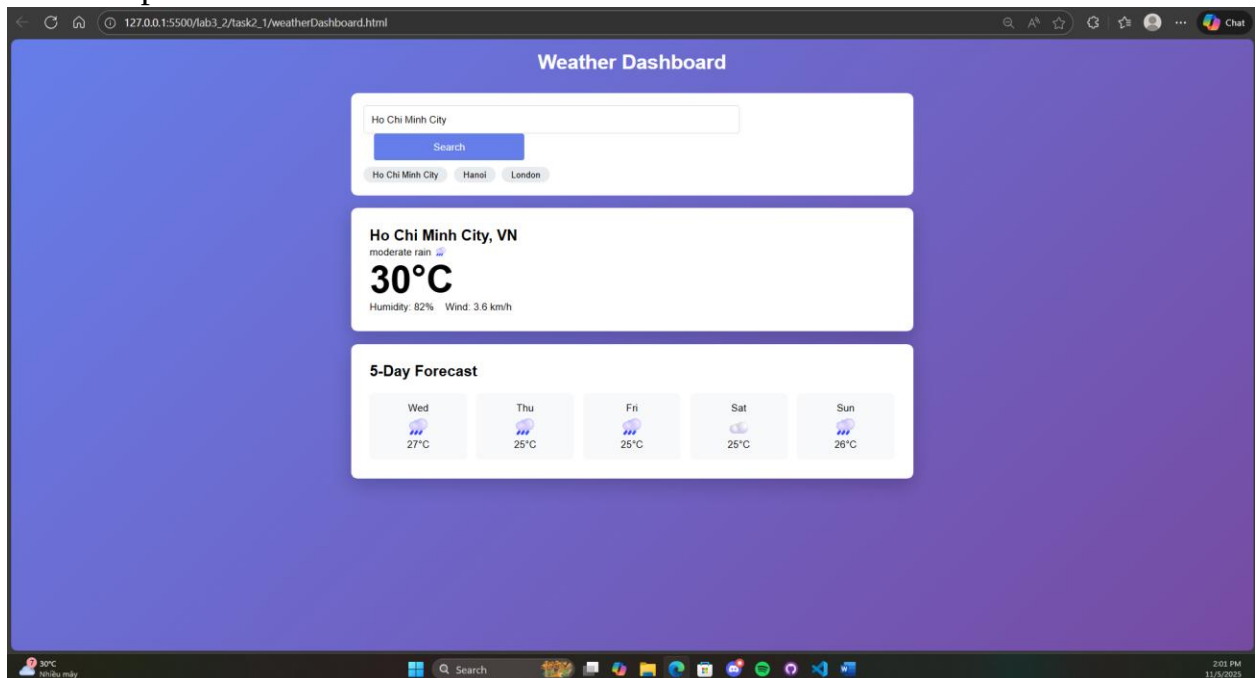
```
function loadRecentSearches() {
  // TODO: Load and display recent searches
  const recent = JSON.parse(localStorage.getItem('recentCities')) || [];
  const container = document.getElementById('recentSearches');
  container.innerHTML = recent.map(
    city => `<div class="recent-city" onclick="searchFromRecent('${city}')">${city}</div>`
  ).join('');
}
```

- showError(message) / clearError()
 - Handles error messages.
 - showError() displays an alert-like red box.
 - clearError() removes any existing error before a new search.

```
function showError(message) {
  // TODO: Display error message
  document.getElementById('errorMessage').innerHTML = `<div class="error">${message}</div>`
}

function clearError() {
  // TODO: Clear error message
  document.getElementById('errorMessage').innerHTML = '';
}
```

- Final output of the web:



II. Task 2.2

- searchRepositories(query, sort, page)

- This function sends a request to the GitHub API using the entered search keyword.
- It builds the URL in the format:
[https://api.github.com/search/repositories?q=\\${query}&sort=\\${sort}&page=\\${page}&per_page=10](https://api.github.com/search/repositories?q=${query}&sort=${sort}&page=${page}&per_page=10)
- Shows a “Loading...” message before fetching results.
- If the request succeeds, it returns the list of repository data (data.items).
- If something goes wrong (like a rate limit error), it calls showError() to display a message

```

async function searchRepositories(query, sort = 'stars', page = 1) {
  // TODO: Fetch repositories from GitHub API
  // URL: `https://api.github.com/search/repositories?q=${query}&sort=${sort}&page=${page}&per_page=10`
  try {
    clearError();
    document.getElementById('repoList').innerHTML = '<div class="loading">Loading...</div>';
    const response = await fetch(`https://api.github.com/search/repositories?q=${query}&sort=${sort}&page=${page}&per_page=10`);

    if (!response.ok) {
      if (response.status === 403) {
        throw new Error('Rate limit exceeded. Please try again later.');
      } else {
        throw new Error('Failed to fetch repositories.');
      }
    }

    const data = await response.json();
    totalResults = data.total_count;
    return data.items;
  } catch (error) {
    showError(error.message);
    return [];
  }
}

```

- displayRepositories(repos, append = false)
 - Displays the repository search results on the webpage.
 - If append is false, it clears any previous results (used for new searches).
 - If no repositories are found, it shows a message saying “No repositories found.”
 - Otherwise, it loops through the results and adds cards generated from createRepoCard().
 - If more results exist, it shows a “Load More” button to view the next page of repositories.

```
function displayRepositories(repos, append = false) {
  // TODO: Display repository cards
  // If append is true, add to existing results
  // Otherwise, replace all results
  const repoList = document.getElementById('repoList');
  if (!append) repoList.innerHTML = '';

  if (repos.length === 0) {
    repoList.innerHTML = '<p>No repositories found.</p>';
    document.getElementById('loadMoreContainer').innerHTML = '';
    return;
  }
}
```

```
    repos.forEach(repo => {
      repoList.innerHTML += createRepoCard(repo);
    });

    if (repos.length === 10 && currentPage * 10 < totalResults) {
      document.getElementById('loadMoreContainer').innerHTML = `
        <div class="load-more">
          <button onclick="loadMore()">Load More</button>
        </div>`;
    } else {
      document.getElementById('loadMoreContainer').innerHTML = '';
    }
  }
}
```

- createRepoCard(repo)
 - Creates a single repository card using template literals.
 - Each card displays:
 - The repository name (clickable link)
 - Description text
 - Star count ★
 - Fork count 🍴
 - Owner name 👤
 - Language badge if available
 - If the repo has no description, it shows “No description provided.”


```

function createRepoCard(repo) {
  // TODO: Create HTML for a single repository card
  // Include: name, description, stars, forks, language
  // repo.name, repo.description, repo.stargazers_count
  // repo.forks_count, repo.language, repo.html_url, repo.owner.login
  return `
    <div class="repo-card">
      <a href="${repo.html_url}" target="_blank" class="repo-name">${repo.name}</a>
      <p class="repo-description">${repo.description || 'No description provided.'}</p>
      <div class="repo-meta">
        <span>★ ${formatNumber(repo.stargazers_count)}</span>
        <span>🔗 ${formatNumber(repo.forks_count)}</span>
        <span>👤 ${repo.owner.login}</span>
        ${repo.language ? `<span class="language-badge">${repo.language}</span>` : ''}
      </div>
    </div>
  `;
}

```

- performSearch()
 - Triggered when the user clicks the Search button or presses Enter.
 - Gets the search input and sort option from the dropdown.
 - If the search box is empty, it shows an error message.
 - Otherwise, it resets the page number to 1 and fetches the first 10 repositories using searchRepositories().
 - Then it calls displayRepositories() to show the results.

```

async function performSearch() {
  // TODO: Get search query and sort option
  // TODO: Reset page to 1
  // TODO: Call searchRepositories
  // TODO: Display results
  const query = document.getElementById('searchInput').value.trim();
  const sort = document.getElementById('sortSelect').value;
  if (query === '') {
    showError('Please enter a search term.');
    return;
  }

  clearError();
  currentQuery = query;
  currentPage = 1;

  const repos = await searchRepositories(query, sort, currentPage);
  displayRepositories(repos);
}

```

- loadMore()

- Triggered when the Load More button is clicked.
- Increases the currentPage value by 1 to move to the next set of results.
- Calls searchRepositories() again to fetch more repositories and appends them using displayRepositories(repos, true).

```
async function loadMore() {
  // TODO: Increment page
  // TODO: Fetch next page
  // TODO: Append results
  currentPage++;
  const sort = document.getElementById('sortSelect').value;
  const repos = await searchRepositories(currentQuery, sort, currentPage);
  displayRepositories(repos, true);
}
```

- showError(message)
 - Displays an error message in a red box at the top of the page.
 - Used when the API request fails, or the user leaves the search box empty.

```
function showError(message) {
  // TODO: Display error message
  document.getElementById('errorMessage').innerHTML = `<div class="error">${message}</div>`;
}
```

- clearError()
 - Clears any previous error messages before showing new results.
 - Prevents old error messages from staying on the page during new searches.

```
function clearError() {
  // TODO: Clear error message
  document.getElementById('errorMessage').innerHTML = '';
}
```

- Final output:

